



FPGA prototyping System TX Report

Student Names	
Ahmed Hossam Rafik	
Ahmed Haitham Othman	
Mohamed Ahmed Mohamed Hussien	
Mohamed Adel Abdelrahem	
Abdelrhman Khaled Fouad	
Youssef Mohamed Mamdouh	
Moustafa Saad Dawood	

Implementation and Verification Report: TX Subsystem

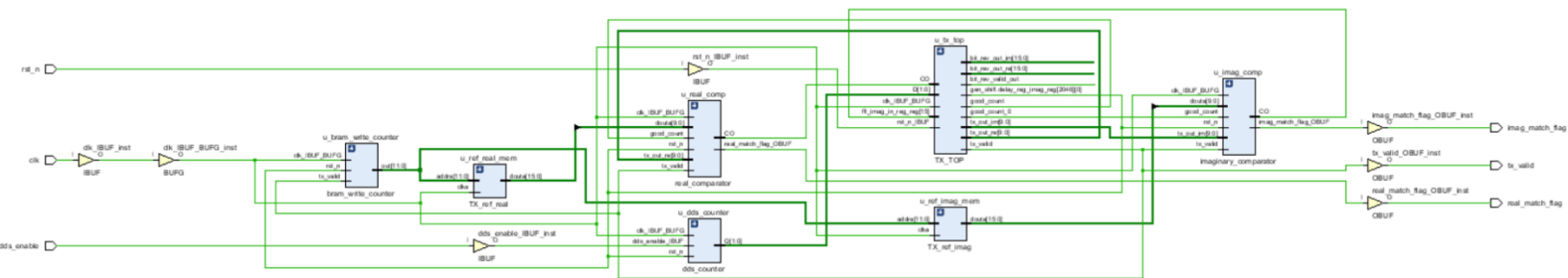
Table of Contents

1. System Architecture & Synthesis Design
2. Memory Allocation Strategy
3. Timing Analysis & Critical Path Optimization
4. Resource Utilization & Device Layout
5. Functional Simulation & Verification
6. Physical Implementation & Hardware Verification

1. System Architecture & Synthesis Design

The top-level synthesis design consists of a test wrapper that encapsulates the main TX processing path and the verification infrastructure. The architecture is divided into the following key components:

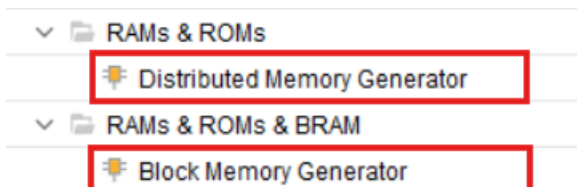
- **TX Core:** The primary data path responsible for processing the input signals.
- **Direct Digital Synthesizer (DDS):** Generates the input stimulus, driven by a configuration register and an initialization counter.
- **Reference Memories:** Two distinct ROMs used to store the golden reference data for the real and imaginary outputs.
- **Verification Comparators:** Two separate comparators evaluate the real and imaginary outputs of the TX core against the reference memories to assert match flags.



2. Memory Allocation Strategy

Due to the heavy reliance on memory within the design, strategic allocation was required to prevent Block RAM (BRAM) over-utilization. The architecture leverages a hybrid memory approach:

- **Block Memory Generator:** Applied to structures requiring larger depths to efficiently utilize dedicated BRAM tiles.
- **Distributed Memory Generator:** Applied to smaller depth requirements. This forces Vivado to map the memory into LUTRAMs (Look-Up Table RAM) rather than consuming valuable and limited BRAM resources.



3. Timing Analysis & Critical Path Optimization

Initial implementation attempts yielded a massive 20ns setup timing violation. This was traced back to the FFT stages being fully combinational, causing excessive delay through multiple chained adders and multipliers.

To achieve timing closure (positive setup and hold slack), the following optimization steps were executed:

- **Pipelining the Data Path:** The combinational stages were converted to sequential stages. This added 12 cycles of pipeline latency, bringing the total system latency to $N + 12$ cycles, effectively breaking up the critical path.
- **Retiming Enable:** Vivado's physical synthesis retiming was enabled to automatically move flip-flops forward or backward across combinational logic to balance delays.
- **Performance Strategy:** The Vivado implementation strategy was set to Performance_High (perfhgh) to prioritize timing over area.
- **Resource Sharing Disabled:** Disabled resource sharing to prevent the synthesis tool from multiplexing logic, trading an increase in area for faster dedicated data paths.
- **LUT Combining Disabled (-no_lc):** The -no_lc flag was applied to prevent Vivado from combining lower-input LUTs into higher-input LUTs. While combining saves area, it increases routing delay; disabling it ensured the fastest possible logic transitions.

Report Options

Strategy:  Vivado Synthesis Default Reports (Vivado Synthesis 2018) ▼

Options

Description: Higher performance designs, resource sharing is turned off, the global fanout

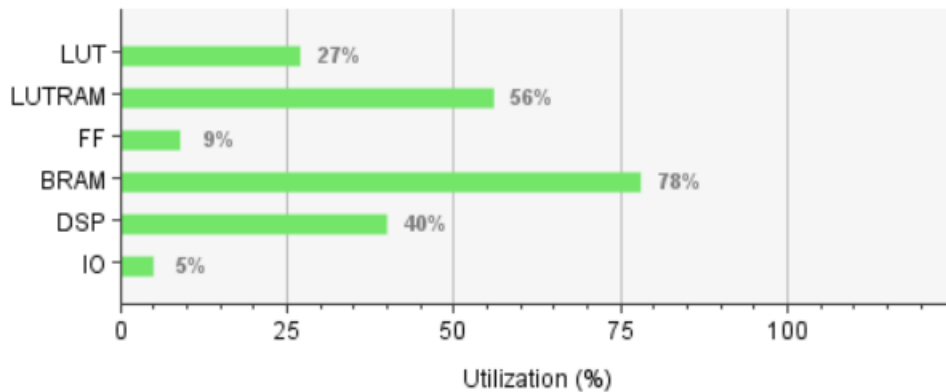
▼ Synth Design (vivado)

tcl.pre		...
tcl.post		...
-flatten_hierarchy	full	▼
-gated_clock_conversion	off	▼
-bufg	12	
-fanout_limit*	400	
-directive	Default	▼
-retiming	☒	
-fsm_extraction*	one_hot	▼
-keep_equivalent_registers*	☒	
-resource_sharing*	off	▼
-no_lc*	☒	

4. Resource Utilization & Device Layout

Following the aggressive timing optimization strategies, the design successfully implemented on the target FPGA. The device layout shows the logic clustered across multiple clock regions (X0Y0, X1Y1, etc.). The final resource utilization confirms the success of the hybrid memory strategy, keeping BRAM usage within acceptable limits while heavily leveraging LUTRAM:

Resource	Utilization	Available	Utilization %
LUT	14627	53200	27.49
LUTRAM	9666	17400	55.55
FF	9371	106400	8.81
BRAM	109	140	77.86
DSP	88	220	40.00
IO	6	125	4.80



5. Functional Simulation & Verification

The dynamic simulation confirms the pipeline operates as intended from initialization to continuous data streaming. The verification sequence proceeds as follows:

1. Three configuration values are passed cycle-by-cycle from the configuration register to initialize the DDS.
2. Once configured, the dds_enable signal asserts high.
3. The generated stimulus flows block-by-block through the TX pipeline, ultimately reaching the final bit-reversal stage.
4. The tx_valid signal asserts high, indicating valid output data is available.
5. The internal counters synchronize the TX output with the reference memory outputs.
6. The real_match_flag and imag_match_flag both assert high simultaneously, confirming that the majority of the hardware-calculated values perfectly match the golden reference.

